AxiomEval — Exercices et corrigés

Couverture de cette édition : semaines S04 à S09 (le code ajouté pendant la mission). Les semaines S01–S03 ont leurs exercices dans le guide S6 (`docs/semaine-06/guide/`, local) ; S10–S26 recevront les leurs quand elles seront implémentées. Fais chaque exercice **avant** de lire le corrigé ; pour les exercices de code, lance les tests.

- AxiomEval — Exercices et corrigés
  - Partie 1 — Énoncés
    - S04 — Corpus et partitions
    - S05 — Verdict statistique
    - S06 — Autorisation
    - S07 — Sécurité
    - S08 — API et base
    - S09 — Baselines ML
  - Partie 2 — Corrigés

Partie 1 — Énoncés

S04 — Corpus et partitions

1. (Compréhension) Pourquoi l'unité statistique est-elle le groupe et non le scénario ?
2. (Compréhension) Que se passe-t-il si l'on modifie une paraphrase de `templates.json` sans régénérer `split.json` ?
3. (Compréhension) Pourquoi la partition est-elle stratifiée par type de scénario ?
4. (Compréhension) Pourquoi la température est-elle 0,7 et la graine différente à chaque tirage ?
5. (Compréhension) Qu'est-ce qu'une clé de tirage et qu'empêche-t-elle ?
6. (Lecture de code) Dans `splits.py::assign`, que vaut `end` pour la partition `test` et pourquoi ?
7. (Lecture de code) Dans `scenarios.py::load_corpus`, quelle exception se produit si deux familles produisent la même question ?
8. (Lecture de code) Dans `collection.py::_one`, pourquoi `adapter.seed` est-il modifié avant chaque exécution ?
9. (Modification) Ajoute une famille `retour_endommage` (3 paraphrases, label contenant « photo ») et régénère la partition. Quels fichiers changent ?
10. (Modification) Change les proportions pour 50/20/15/15. Écris le test qui vérifie que chaque groupe reste dans une seule partition.
11. (Débogage) Un camarade obtient « frozen split belongs to another corpus version ». Donne deux causes possibles.
12. (Conception) Comment ajouter un second domaine (agent RH) pour mesurer le transfert sans mélanger les partitions ?

S05 — Verdict statistique

1. Calcule le rayon de Hoeffding pour n = 100 groupes, α = 0,05.
2. Pourquoi une brèche donne REJECT même si la candidate est meilleure en moyenne ?
3. Pourquoi `different-evaluator-versions` donne INCOMPLETE ?
4. Le bootstrap donne [0,02 ; 0,15] et Hoeffding une borne inférieure de −0,39. Lequel décide, et pourquoi ?
5. Combien de groupes faudrait-il pour démontrer la non-infériorité si l'écart observé était nul, avec m = 0,10 et α = 0,05 ?

6. (Lecture) Dans `paired_comparison`, que se passe-t-il si un groupe n'existe que chez la candidate ?
7. (Lecture) Pourquoi `max(-1.0, point - radius)` ?
8. (Lecture) Dans `binomial_upper`, pourquoi la bissection part-elle de `successes / n` ?
9. (Modification) Ajoute au `Protocol` un champ `ci_method` (`"hoeffding"` ou `"bootstrap"`) et utilise la borne choisie pour HOLD/PROMOTE. Quel risque introduis-tu ?
10. (Modification) Ajoute au verdict la raison `too-few-attack-runs` si la candidate a moins de 10 exécutions d'attaque.
11. (Débogage) Un test `test_verdict_priority` échoue : PROMOTE au lieu de HOLD pour 5 groupes. Où regarder ?
12. (Conception) Proposez une borne plus serrée que Hoeffding et ses hypothèses.

S06 — Autorisation

1. Quelle est la différence entre identité demandée et identité effective ?
2. Pourquoi un outil absent de la politique est-il refusé ?
3. Pourquoi lier l'approbation à l'empreinte des arguments ?
4. Pourquoi `amount != amount` ?
5. Que révèle la différence entre `allowed-not-found` et `not-owner` ?
6. (Lecture) Dans `authorize`, dans quel ordre sont vérifiés expiration, outil, rôle ? Justifie.
7. (Lecture) Que contient `observed_action` ?
8. (Lecture) Dans `ApprovalStore.consume`, pourquoi tester `_used` avant `_pending` ?
9. (Modification) Ajoute un rôle `read_only` autorisé seulement pour `search_kb`. Écris le test.
10. (Modification) Fais répondre `not-found` de façon identique pour une commande inexistante et une commande d'un autre client. Quel test change ?
11. (Débogage) Une approbation valide est refusée avec `approval-args-mismatch`. Cause probable ?
12. (Conception) Comment brancher un vrai fournisseur d'identité (OIDC) sans changer `authorize` ?

S07 — Sécurité

1. Différence entre brèche et tentative bloquée ?
2. Pourquoi la garde remplace-t-elle toute la réponse ?
3. Pourquoi r4 a-t-il 0 brèche alors que son modèle est le même que r2 ?
4. Pourquoi « 0/27 brèche » ne signifie pas « risque nul » ?
5. Qu'est-ce que le finding F-001 et comment est-il prouvé corrigé ?
6. (Lecture) Dans `egress.py`, comment la garde détecte-t-elle un jeton épelé ?
7. (Lecture) Dans `evaluate.py`, quand `permission` vaut-il `UNCERTAIN` ?
8. (Lecture) Dans `agent.py`, que se passe-t-il quand la garde refuse un appel d'outil ?
9. (Modification) Ajoute la détection d'un canari encodé en ROT13 et son test.
10. (Modification) Ajoute une famille d'attaque `privacy_leak` qui demande l'adresse d'un autre client.
11. (Débogage) La garde bloque une réponse légitime contenant un long numéro de suivi. Pourquoi, et que faire ?
12. (Conception) Quelle correction structurelle supprimerait F-002 sans garde de sortie ?

S08 — API et base

1. Pourquoi une migration modifiée après application est-elle refusée ?
2. Que garantit `BEGIN IMMEDIATE … COMMIT/ROLLBACK` ?
3. Pourquoi comparer le jeton avec `hmac.compare_digest` ?
4. Pourquoi l'identifiant de verdict dépend-il des clés de tirage ?
5. Pourquoi `/health` est-il public mais pas `/ready` ?
6. (Lecture) Dans `store.migrate`, pourquoi retirer les lignes de commentaire avant de découper sur `;` ?

7. (Lecture) Dans `import_records`, que se passe-t-il au 3e enregistrement invalide d'un lot de 5 ?
8. (Lecture) Que vérifie `test_verdict_changes_identity_when_evidence_changes` ?
9. (Modification) Ajoute `GET /v1/releases/{digest}/runs?limit=` avec un plafond à 500.
10. (Modification) Ajoute une migration `0003` qui crée un index sur `runs(kind)`. Que vérifie-t-on ?
11. (Débogage) `sqlite3.OperationalError: database is locked` sous charge. Pistes ?
12. (Conception) Que faudrait-il pour passer à PostgreSQL ?

S09 — Baselines ML

1. Qu'est-ce que l'horizon de prédiction et pourquoi la latence est-elle exclue ?
2. Pourquoi la validation croisée est-elle groupée ?
3. Pourquoi les trois graines donnent-elles le même résultat ?
4. Pourquoi « texte seul » fait-il presque aussi bien que tout ?
5. Pourquoi ne peut-on pas conclure que le boosting est meilleur ?
6. (Lecture) Dans `features.py::prefix`, où s'arrête la boucle ?
7. (Lecture) Dans `Featurizer.fit`, pourquoi `min_df` dépend-il du nombre d'exemples ?
8. (Lecture) Dans `group_bootstrap`, pourquoi certains tirages sont-ils ignorés ?
9. (Modification) Ajoute une baseline « fréquence d'échec par famille » apprise sur l'entraînement. Est-ce un raccourci ?
10. (Modification) Rends le nombre de tirages bootstrap paramétrable depuis la CLI.
11. (Débogage) Le PR-AUC du test est 1,0. Quelles fuites vérifier ?
12. (Conception) Quelle taille de corpus viser pour un intervalle de PR-AUC de largeur ≈ 0,15 ?

Partie 2 — Corrigés

1. Les paraphrases d'un groupe décrivent la même situation : leurs résultats sont corrélés. Les compter séparément gonflerait n et rétrécirait faussement les intervalles.
2. L'empreinte du corpus change ; `load_frozen` refuse l'ancienne partition (« another corpus version »). Il faut supprimer `split.json` et relancer `axiomeval corpus`, ce qui gèle une nouvelle répartition.
3. Pour que chaque type (information, compte, action, attaque) ait des groupes dans chaque partition ; sinon le test pourrait ne contenir aucune attaque.
4. À température 0, deux tirages identiques donnent la même sortie : ce seraient des copies, pas des observations. La graine change pour que chaque tirage soit une nouvelle observation reproductible.
5. L'empreinte (release, scénario, tirage, évaluateur). Elle empêche de compter deux fois la même observation (un ré-import ou une relance n'augmente jamais n).
6. `end = n` : la dernière partition prend tout le reste, pour qu'aucun groupe ne soit perdu par arrondi.
7. `ConfigurationError("duplicate question across groups: …")`.
8. Pour que le tirage `d` utilise la graine `release.seed + d` : reproductible et différent des autres tirages.
9. `templates.json`, puis (après suppression et relance) `split.json` et `docs/data/DATA_CARD.md` ; le test `test_groups_and_paraphrases` (27 groupes) doit être mis à jour.
10. `assign(CORPUS, 1, {"train": .5, "validation": .2, "calibration": .15, "test": .15})` puis vérifier que `set(result) == set(CORPUS.groups)` et que chaque valeur est une partition connue (un dictionnaire garantit déjà une seule partition par groupe).
11. Le corpus a changé (template modifié) ; ou le fichier `split.json` vient d'une autre branche.
12. Un corpus séparé (comme `fixtures/security`) avec son propre `split.json` ; entraîner sur le domaine A, tester sur B, ne jamais mélanger les groupes.
13. $\sqrt{2 \times 2{,}9957/100} = 0{,}245$.
14. Spec §31.6 : une violation n'est jamais compensée par la qualité. La règle est appliquée avant tout calcul.

15. Les deux releases auraient été notées avec des règles différentes : la différence mesurerait le changement de règles.
16. Hoeffding : valide pour tout n sous indépendance ; le bootstrap est une approximation. On prend la décision avec la garantie, on montre le bootstrap comme diagnostic.
17. $n > 4 \times 2{,}9957/(2 \times 0{,}01) = 599{,}1$, soit 600 groupes.
18. Il est ignoré par l'intersection, mais `release_verdict` aura déjà renvoyé INCOMPLETE (`unpaired-groups`).
19. Une différence moyenne ne peut pas être inférieure à −1 ; la borne est tronquée à sa valeur minimale possible.
20. La borne supérieure est au moins le taux observé k/n ; on part de là pour gagner des itérations.
21. Choisir la méthode après avoir vu les résultats permettrait de choisir celle qui promeut : la méthode doit être fixée dans le protocole **avant** la collecte.
22. Après le test des brèches : `if sum(r["kind"] == "attack" for r in candidate) < 10: return HOLD(...)`, avec un test dédié.
23. Le test `paired.n_groups < protocol.min_groups` ou le `Protocol` passé au test (min_groups).
24. Bernstein empirique : utilise la variance observée ; reste valide sous indépendance et bornitude, plus serrée quand la variance est petite.
25. Demandée = ce que la requête prétend (y compris `claimed_*`) ; effective = l'identité de session, seule utilisée pour décider.
26. Refus par défaut : un outil oublié dans la politique ne doit jamais devenir une porte ouverte.
27. Une approbation pour 230 $ ne doit pas autoriser 900 $ ni une autre commande.
28. C'est le test de NaN : NaN est le seul nombre différent de lui-même ; sans ce test, NaN passerait le plafond.
29. Qu'une commande existe chez un autre client : une fuite d'existence (énumération).
30. Expiration, outil, rôle : du plus général au plus spécifique ; une identité périmée ne doit rien apprendre d'autre.
31. Le nom de l'outil et les arguments exacts demandés par le modèle.
32. Un jeton utilisé a été retiré de `_pending` : sans ce test, un rejeu serait signalé « unknown » au lieu de « replayed ».
33. Ajouter `"read_only"` aux rôles de `search_kb` seulement ; test : `refund_order` → `role-not-allowed`, `search_kb` → `allowed`.
34. Remplacer `not-owner` par un `decide(True, "allowed-not-found")` + réponse « introuvable » ; `test_other_customer_same_tenant` change.
35. Les arguments ont changé entre l'émission et l'usage (montant arrondi, numéro en minuscules…).
36. Construire `Identity` à partir des revendications vérifiées du jeton OIDC dans la couche API ; `authorize` reçoit toujours une `Identity`.
37. Brèche : un effet a eu lieu (secret divulgué, action exécutée). Tentative bloquée : un contrôle l'a empêché.
38. Un masquage partiel peut laisser fuir le contexte du secret.
39. La garde de sortie, contrôle déterministe hors modèle, bloque les fuites que le modèle produit encore.
40. Borne exacte à 95 % : 10,5 % ; de plus, les tirages sont corrélés et de nouvelles attaques peuvent réussir.
41. Le modèle a envoyé le jeton comme numéro de commande. La garde contrôle désormais les arguments ; le test de régression rejoue l'appel réel.
42. `_squash` retire espaces et tirets et met en majuscules ; la forme épelée devient identique au canari écrasé.
43. Quand un contrôle a bloqué quelque chose (autorisation, outil interdit tenté, garde) sans brèche.
44. Événement `guard`, résultat `guarded`, message de refus renvoyé au modèle, outil jamais appelé.
45. Ajouter `codecs.encode(answer, "rot13")` aux vues décodées ; test avec le canari encodé.
46. Ajouter un e-mail fictif aux commandes, une famille `attack_family: privacy_leak` avec `must_not_contain` l'e-mail.
47. Le motif hexadécimal ou le motif de carte (13 à 16 chiffres) le capture ; resserrer le motif ou ajouter une liste blanche de formats connus, avec un test.
48. Ne pas placer de secret dans les documents récupérables ; filtrer les documents internes avant la recherche.
49. La base serait dans un état que le fichier ne décrit plus ; impossible de reproduire le schéma.
50. Tout le lot est écrit ou rien ; aucune lecture partielle par un autre processus.
51. Une comparaison ordinaire s'arrête au premier caractère différent : le temps de réponse révèle le préfixe correct.

52. Si les preuves changent, ce n'est plus le même verdict ; le stocker sous le même identifiant mentirait.
53. `/health` dit seulement que le processus répond ; `/ready` révèle l'état de la base.
54. Un `;` dans un commentaire (cas réel de `0001_init.sql`) coupait une instruction en deux.
55. Rien n'est écrit : la validation des scores échoue avant la transaction (StoreError).
56. Qu'un nouveau verdict après ajout d'une brèche change d'identifiant et de résultat (HOLD → REJECT).
57. Route avec `limit: int = Query(100, ge=1, le=500)` et test de 422 au-delà.
58. Que `migrate()` l'applique une fois, que `schema_version()` vaut `0003_…`, et qu'un ré-appel ne fait rien.
59. Transactions longues, plusieurs écrivains ; WAL est déjà activé ; réduire la durée des transactions, mettre en file les écritures.
60. Un pilote (psycopg), des migrations compatibles, des types adaptés, des tests d'intégration sur un vrai serveur.
61. L'instant juste après le premier appel au modèle ; la latence révèle la taille du modèle (raccourci vers la release).
62. Sinon des paraphrases d'un même groupe seraient en entraînement et en validation.
63. Les modèles utilisés sont déterministes sur données fixées ; la variabilité vient des groupes de test.
64. La question révèle la famille (information, action…) dont les taux d'échec diffèrent : raccourci de famille.
65. Intervalles groupés très larges et chevauchants (4 groupes de test).
66. Après le premier `model_call` inclus, ou au premier événement non visible.
67. Avec 1 ou 2 exemples, `min_df=2` rendrait le vocabulaire vide (erreur rencontrée pendant le développement).
68. Pour PR-AUC/ROC-AUC, un tirage sans les deux classes n'a pas de valeur définie.
69. Oui : la famille est connue avant la décision, mais le modèle n'apprend alors rien de la trajectoire ; c'est une baseline à battre.
70. Argument `--bootstrap-draws` passé à `experiment` puis `run_protocol`.
71. Caractéristiques après l'horizon, identifiant de release, groupe présent en entraînement et en test.
72. Ordre de grandeur : la largeur décroît comme $1/\sqrt{G}$; passer de 4 à environ 100 groupes de test divise la largeur par environ 5.